AGORA — Official Feature List

Agentic Generator and OCR-based Recognition engine platform

October 10, 2026

Overview

AGORA is a web platform where teachers create, collect and check written, objective and coding assessments, plan lessons, and track class and student performance, with AI assisting and the teacher always able to review. It is built for colleges and universities: a class is a course offered in a semester, and students enroll course by course rather than in a fixed homeroom section. Its full title is *AGORA: Agentic Generator and OCR-based Recognition engine platform*.

Users: Super/Master Admins, College Admins, Teachers (faculty and instructors) and Students.

Design principles

- **Teacher in the loop.** AI scores are suggestions the teacher can review and override. Low-confidence OCR words are flagged for human review, and final grading is never fully autonomous.
- **Accounts are provisioned, not self-registered.** Colleges, admins and teachers are created from the top down.
- **Cheap, deterministic checking where possible.** Objective items are checked by direct answer-key comparison, with no model involved.
- **Local and open models.** Language models run locally (Qwen 2.5 via Ollama) and the checker models are open-weight, so there is no per-call billing and no student data goes to third-party APIs.

Features at a glance

#	Feature	What it does
1	Account Registration & Provisioning	Top-down creation of colleges, college admins and teacher accounts with role-based access
2	Online Classroom Management	Assigned classes (a course offered in a semester) where teachers manage enrolled students, assessments and class tracking
3	Sandbox: Manual Template Mapping	Teacher-defined, reusable answer-sheet layouts with answer keys
4	OCR & NLP Essay / Short-Answer Scorer	Scores handwritten (scanned) or typed written answers
5	AI Agentic Console	Generates, secures and distributes quizzes from natural-language instructions
6	ACTS NoteBook	Generates Outcome-Based Education lesson plans from lecture materials and learning goals or course outcomes
7	Digital E-Classrecord	Term-based grade ledger with weighted, automatic computation and conversion to the college grading scale
8	Class Performance Analytics	Per-class dashboards of averages, distributions and missed items, with department and program roll-ups
9	Student Retention Analytics	4-quadrant matrix of grades against attendance, with college-set thresholds, to flag at-risk students
10	Online Code Compiler / Code Checker	Runs and grades student-submitted Python code against test cases
11	External API & Data Architecture	Hosted model API for outside platforms, plus the shared data layer

Feature 1: Account Registration & Provisioning

Core, security-critical. Every account in AGORA is created by an authority above it, and there is no public sign-up anywhere in the system. This is the entry point for every other feature, since nothing is reachable without an account and a defined role.

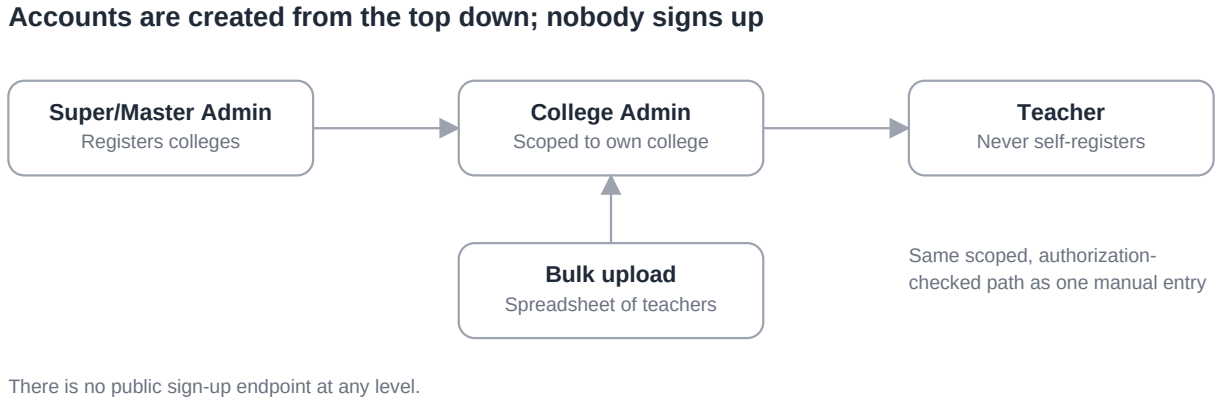


Figure 1. Provisioning hierarchy: accounts are created top-down and never self-registered.

Accounts follow a top-down hierarchy:

1. A **Super/Master Admin** registers and manages the participating colleges.
2. A **College Admin** (for example a registrar, dean or program head), scoped strictly to their own college, creates or bulk-uploads teacher accounts for that college. The scoping is enforced structurally, so cross-college account creation is not possible.
3. **Teachers** receive accounts from their College Admin and never self-register.

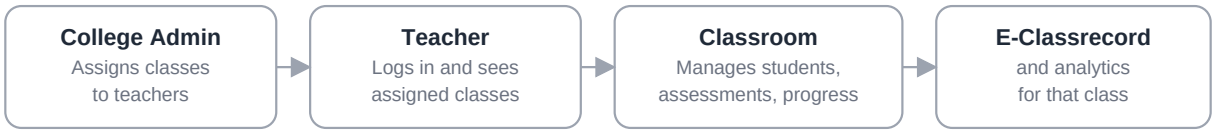
This mirrors how colleges onboard staff in practice: the college already has the definitive list of who is employed to teach, whether full-time, part-time or visiting. It also removes impersonation, spam registration and unverified-identity risks, since there is no open sign-up form to defend.

- **Bulk upload.** A College Admin can onboard many teachers at once from a spreadsheet of teacher records (including each teacher's department, rank and employment status). Bulk entries go through the same scoped, authorization-checked path as a single manual entry.
- **College records on accounts.** Teacher accounts are tied to an academic department; student accounts carry a student number, a degree program and a year level, so classes, rosters and analytics can be scoped correctly.
- **Role-based access control.** Each role sees only the modules and data it is authorized for. For example, E-Classrecord and analytics are limited to administrators and the faculty assigned to the class.
- **Credential issuance.** Passwords are stored in encrypted form and must meet minimum security standards. Default or temporary passwords are used during development, and a forced password reset on first login or a short-lived, single-use invite link is planned before deployment.
- **Design reference.** The account-creation approach follows the stars-sys-sync (jerome branch) codebase.

Feature 2: Online Classroom Management

Core. This is the workspace where a teacher spends most of the day. After the college creates the teacher's account (Feature 1), the teacher logs in and sees the classes assigned to them this semester rather than starting from an empty account.

Classes are assigned by the college and feed grades and analytics



Strict rosters: a teacher sees only the students enrolled in their assigned classes.

Figure 2. Classroom assignment and use.

- **Assigned, not self-created.** Classes are assigned to teachers by the college, following the same top-down logic as account provisioning. Each class is one course from the college's catalog offered in one semester, with a section code, schedule and room. The college already knows which teacher handles which course section, so this avoids duplicate, unauthorized or inconsistent class records.
- **Strict rosters.** College students enroll course by course, so each student has a separate enrollment, and often a separate teacher, in every course they take. A teacher manages only the roster of students enrolled in their assigned classes and tracks those specific cohorts.
- **Inside a classroom,** the teacher manages enrolled students, manages the assessments given to that class, and tracks how the class is performing over time.
- **Semester view.** Teacher views and the Central Interface default to the current semester, and earlier semesters stay available for review.
- **Account and catalog management.** Administrators provision and maintain teacher and student accounts, roles and permissions, together with the academic catalog the classes are built from: departments, degree programs, courses and course learning outcomes.
- **Feeds other features.** Classroom results flow into the E-Classrecord (Feature 7) and the analytics dashboards (Features 8 and 9) for that class.
- **Central Interface.** A landing dashboard summarizes active classes, pending assessments, notifications and key system statistics for quick decisions.

Feature 3: Sandbox: Manual Template Mapping

Core. Teachers define their own answer-sheet layout once and reuse it for every exam that shares the format. Departments and instructors print exam sheets in different layouts, with different spacing, question order and answer regions, so a single hardcoded template cannot serve every course.

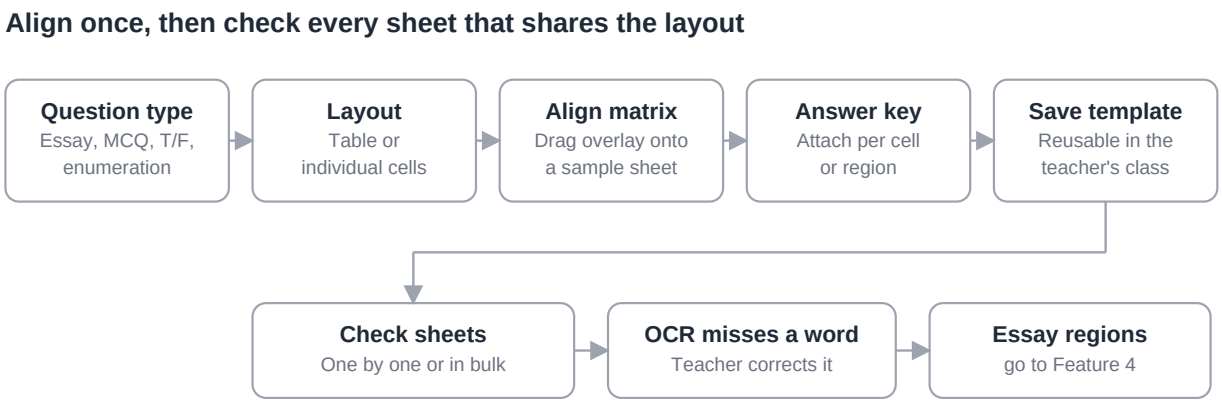


Figure 3. Template creation, reuse and checking workflow.

The one-time setup, per sheet layout:

1 Choose the question type for a region of the sheet: Essay, Multiple Choice, True/False, Enumeration and so on, since each type is read and checked differently.

2 Choose the layout for grid-style items: a table or individual cells.

3 Align the matrix. The teacher drags a matrix overlay on a sample sheet image until it lines up with the printed answer positions.

4 Attach the answer key per cell or region. This lets objective items (MCQ, T/F) be checked by simple answer-key comparison rather than a model.

5 Save as a reusable template within the teacher's classroom. Because a teacher often handles several sections of the same course, a saved template can be duplicated to their other classes for that course.

Once a template exists, checking is repeatable:

- **One-by-one or bulk.** Sheets can be checked individually or processed in bulk.
- **Self-correction.** If OCR fails to recognize a word, the teacher corrects it directly in the interface rather than the system silently guessing or discarding the answer.
- **Scanned sheets feed the scorer.** Essay and short-answer regions go to the OCR & NLP scorer (Feature 4), and the results flow to the E-Classrecord (Feature 7).
- **Design reference.** The approach to template definition, storage and alignment draws on the starscan (feature/with-quizzes) codebase, which solves a similar calibration problem for optical mark recognition sheets.

Feature 4: OCR & NLP Essay / Short-Answer Scorer

Core, highlighted AI feature. This feature automatically scores student-written responses, essays and short answers, and is what makes AGORA AI-driven rather than a digitized version of manual checking. Answers can arrive in two ways, and both end at the same essay checker so the scoring logic is built once.

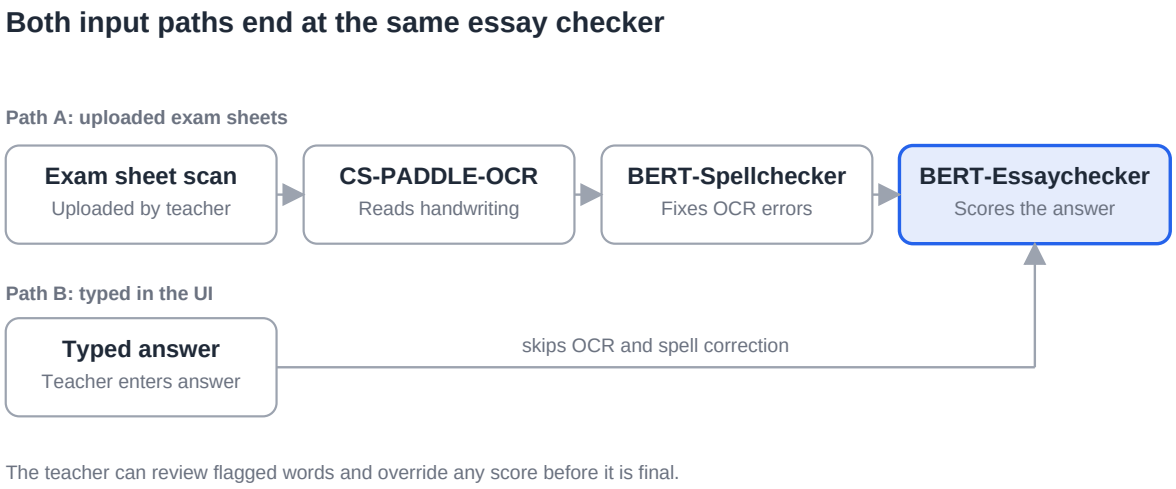


Figure 4. Dual-path scoring pipeline: uploaded sheets and typed answers end at the same essay checker.

Path A: uploaded exam sheets. Many colleges still run paper exams, so the teacher uploads the scanned or photographed answer sheets, typically using a template from Feature 3.

1 **CS-PADDLE-OCR** extracts the handwritten text.

2 **BERT-Spellchecker** corrects OCR misreads before scoring.

3 **BERT-Essaychecker** scores the corrected text.

Path B: manual typing in the UI. The teacher types the student's answer into the interface. This skips OCR and spell correction entirely and goes straight to the essay checker.

The three models

Model	Role	Training
CS-PADDLE-OCR	Reads handwriting from the scanned region. Includes page clean-up, reading-order correction and confidence scoring that flags uncertain words for teacher review.	PaddleOCR recognizer fine-tuned (post-trained) on the IAM Handwriting Database and the team's own handwriting
BERT-Spellchecker	Two-stage OCR text correction: SymSpell fixes spelling and merged or split words, then BERT reranks candidates by sentence context.	BERT fine-tuned on the TU Darmstadt argumentative essays corpus
BERT-Essaychecker	Produces the final score from two sub-scores: writing quality (mechanics) and coverage of the teacher's rubric (content).	DistilBERT mechanics model fine-tuned on the Hewlett Foundation (ASAP) dataset

How the score is built

- **Mechanics sub-score:** a fine-tuned DistilBERT regression model rates writing quality and fluency on a 0 to 1 scale.
- **Content sub-score:** each rubric criterion is compared with the answer's sentences using local sentence embeddings, so paraphrases still match. Criteria are marked met, partial or not met against point values.
- **Teacher-defined rubric:** the teacher supplements the rubric in the UI, writing the criteria and point values for each question.

- **Final score:** a weighted combination of the two sub-scores, 40% mechanics and 60% content by default, shown on a 0 to 10 scale. The weights are adjustable.
- **Short answers** go through the same pipeline and are scored mainly against the teacher's rubric.

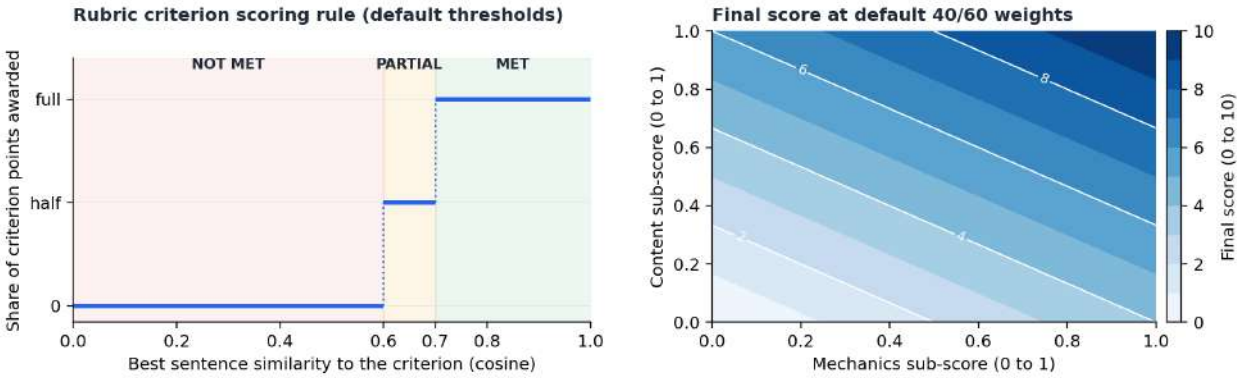


Figure 5. Left: rubric criterion scoring rule. Right: final score as a function of the two sub-scores. Both show the essay checker's default parameters.

Teacher review. The predicted score is always reviewable. The teacher can correct OCR text, check flagged low-confidence words and override any score before it becomes final. Confirmed scores flow to the E-Classrecord (Feature 7).

The full scoring pipeline runs on open models with no paid API calls.

Feature 5: AI Agentic Console

Core, AI-driven. The Agentic Console is a conversational command interface for assessment automation. Instead of filling in forms, a teacher types what quiz they want, and the console generates, formats and securely distributes it using a stateful multi-agent graph.

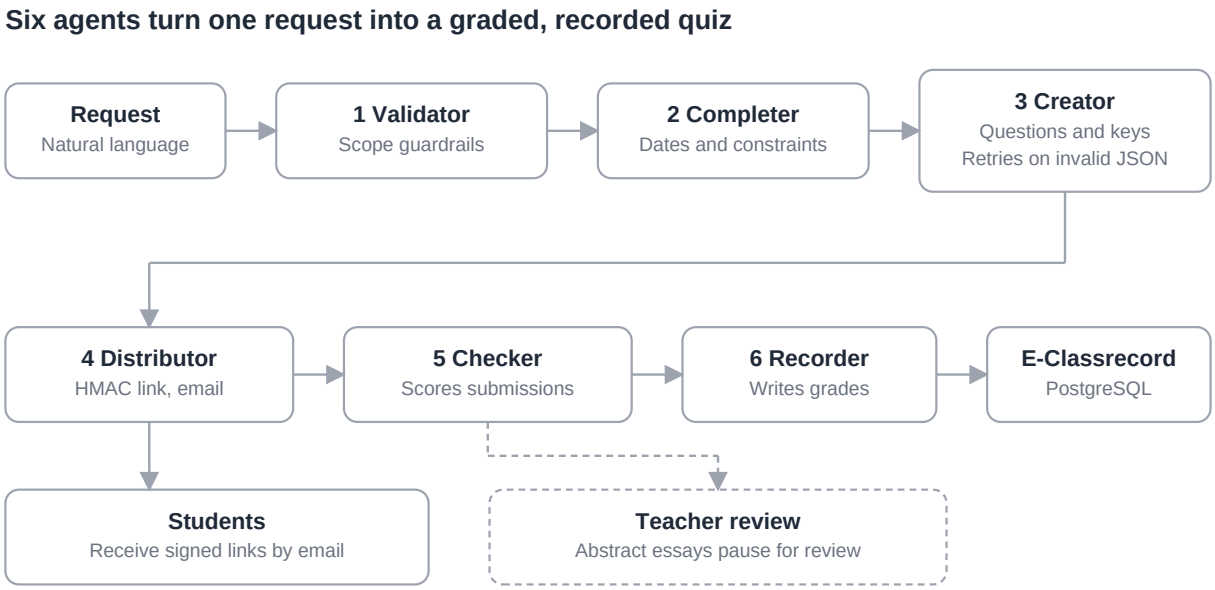


Figure 6. The six-node LangGraph pipeline of the Agentic Console.

What it does

- **Natural-language execution.** Turns unstructured teacher commands into precise generation parameters and database queries.
- **Stateful graph orchestration.** Manages the whole assessment lifecycle through a persistent, non-linear graph. If an agent fails a task, such as invalid JSON, the graph loops back, self-corrects and retries without losing the teacher's original request.
- **Autonomous tool use.** Agents trigger system functions such as date calculation, database queries and cryptographic signing, not just text generation.
- **Secure distribution.** The finished assessment is packaged and sent to the students enrolled in the class through tamper-proof, time-bound links.

The six-node LangGraph pipeline

Node	Agent	Role
1	Validator	Frontline gatekeeper. Sets scope guardrails, rejects non-academic requests and flags missing parameters (topic, difficulty, number of items).
2	Completer	Parses the validated request into exact constraints. Its date tool converts phrases like "due next Friday at 5 PM" into ISO timestamps, and it formats target competencies.
3	Creator	Core generative engine. Drafts the questions, distractors and answer keys, and can retrieve an uploaded document when a quiz is based on a file. A schema-enforcement step forces output into strict JSON, and a conditional edge routes failures back here to retry.
4	Distributor	Attaches an HMAC-SHA256 signature to the assessment link, then emails secure URLs, deadlines and instructions to the class's enrolled students through Gmail SMTP.
5	Checker	Cross-references digital student submissions against AI-generated rubrics, with NLP-based semantic scoring for essay questions.
6	Recorder	Takes the Checker's output and writes the calculated grades into the PostgreSQL E-Classrecord through a database sync tool.

Technologies

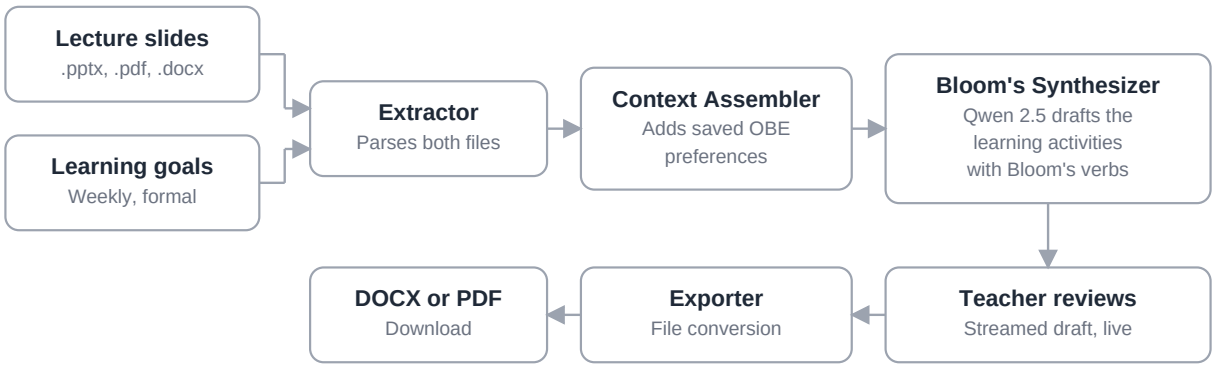
- **LangGraph** is the orchestration backbone: a StateGraph with conditional routing and checkpointers that persist state so the UI can stream live updates.
- **Qwen 2.5 via Ollama** is the local language model, so prompts and academic content never leave the institution and there are no API costs.
- **Python (FastAPI)** is the API gateway and runs the graph asynchronously so the interface stays responsive.
- **React** provides the chat-style console, where teachers watch the graph nodes run in real time and preview the generated JSON assessment.
- **HMAC-SHA256** secures the assessment links against tampering and unauthorized student access.

Teacher in the loop. Highly abstract essay responses are routed to the teacher for review, using LangGraph's interruption and checkpoint capability. Console runs are stored with the teacher's request, model settings and agent outputs.

Feature 6: ACTS NoteBook (Learning Plan Generator)

Core, AI-driven. The ACTS NoteBook is a zero-prompt Outcome-Based Education (OBE) synthesizer. A teacher uploads lecture materials and weekly learning goals, and the NoteBook produces a fully formatted OBE learning plan without any prompt engineering, replacing manual lesson planning. In a college setting the goals usually come from the course's learning outcomes in its syllabus.

Lecture slides and learning goals become a streamed OBE plan



The graph state is ephemeral: lecture files stay in memory for the session only.

Figure 7. ACTS NoteBook processing flow.

What it does

- **Dual-document ingestion.** Extracts text at the same time from two separate inputs: unstructured lecture slides and formal learning goals.
- **Course outcomes.** Learning outcomes saved with the course in the academic catalog (for example CLO1 to CLO5) can be loaded as the learning goals, so a teacher does not retype them for each class or week.
- **Stateful graph execution.** A LangGraph StateGraph keeps the state of the plan while each node reads the extracted text, applies constraints and appends content in order.
- **Taxonomy alignment.** Maps lecture content to the stated goals using Bloom's Taxonomy action verbs, so the plan expresses measurable outcomes.
- **Export.** Converts the approved Markdown draft into a downloadable Word (.docx) or PDF file.

The nodes and their tools

Node	Role
Extractor	Entry point. A document parsing tool (such as PyMuPDF or python-docx) strips text from .pptx, .pdf and .docx files and writes the lecture and goal text to the graph state.
Context Assembler	Fetches the teacher's saved OBE preferences through a database query tool and injects them as hidden system instructions.
Bloom's Synthesizer	The Qwen 2.5 worker. Analyzes how the slides relate to the goals and drafts the sequenced learning activities.
Exporter	Terminal node. Once the teacher approves the streamed draft, a file conversion tool produces the DOCX or PDF.

Bloom's Taxonomy levels used

Level	Typical action verbs
Remembering	define, list, state
Understanding	explain, describe, classify, summarize
Applying	execute, implement, solve, calculate
Analyzing	differentiate, organize, compare, contrast
Evaluating	critique, judge, defend, justify
Creating	design, construct, develop, formulate

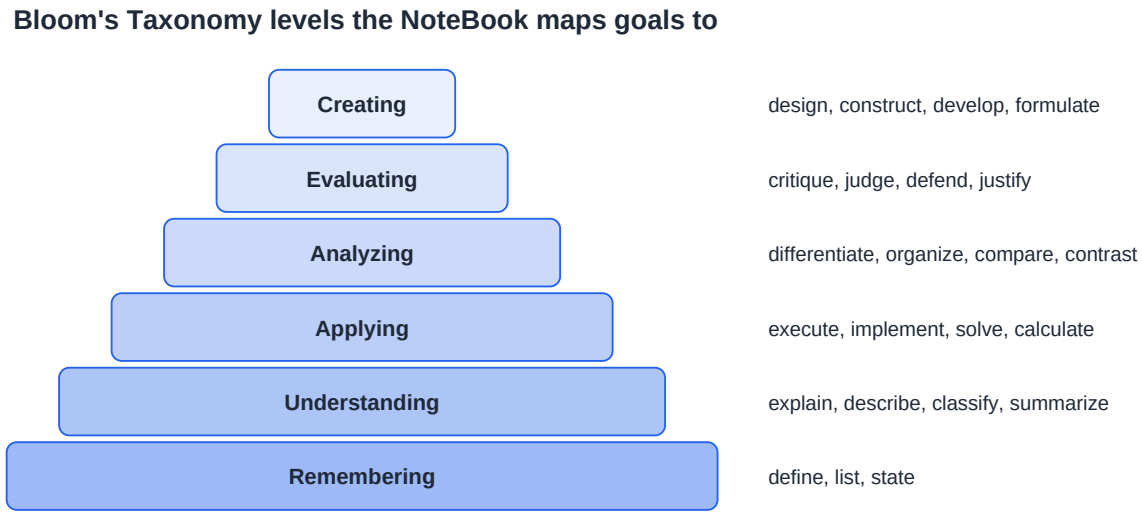


Figure 8. Bloom's Taxonomy levels and action verbs used to align goals.

Technologies

- **LangGraph** manages the data flow between nodes and enables live streaming of state updates as the model drafts.
- **Qwen 2.5 via Ollama** keeps the college's curriculum on-premise.
- **Python (FastAPI)** with WebSockets pushes node transitions to the browser, preventing timeouts during heavy document processing.
- **React** provides a split-screen notebook interface where teachers watch the plan update line by line.

Outputs: a live, editable stream of the learning plan, exportable DOCX and PDF files, and extracted text payloads showing the source materials were parsed. Plans are stored per teacher and class with the topics, source-file references and generated OBE content.

Privacy. The graph state is ephemeral. Uploaded lecture files are extracted into memory only for the length of the WebSocket session, and raw binary files are never permanently stored.

Feature 7: Digital E-Classrecord

Core. The E-Classrecord is the central, dynamic grade ledger. It replaces error-prone manual spreadsheets, automatically ingests AI-graded digital assessments, and gives teachers an interactive grid for physical, offline activities across the whole semester.

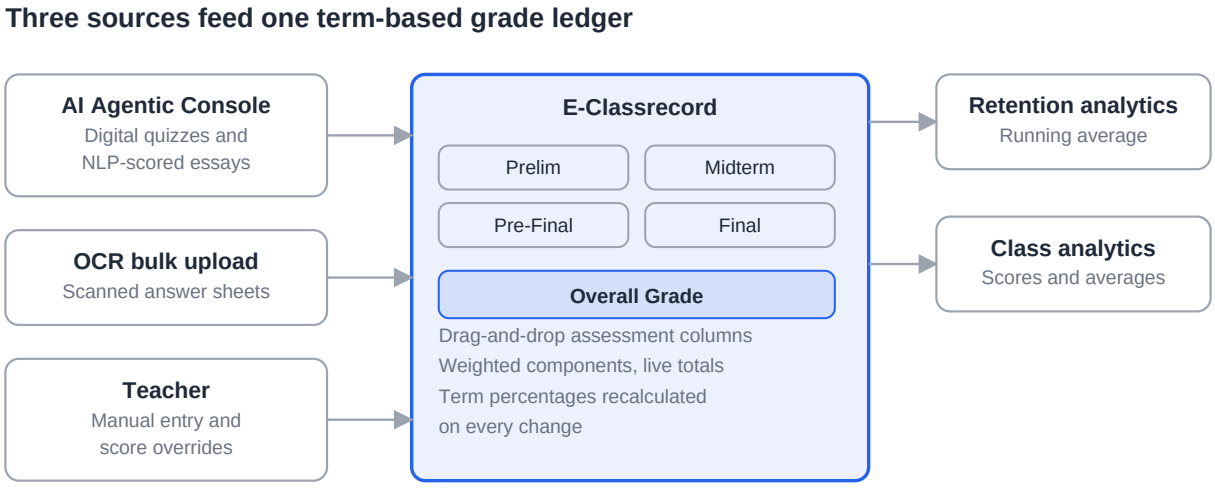


Figure 9. Data sources and outputs of the E-Classrecord.

What it does

- **Multi-term tabs.** Separate tabs for Prelim, Midterm, Pre-Final and Final within the semester, plus a master Overall Grade tab, keep terms apart and prevent calculation overlap.
- **Drag-and-drop columns.** Teachers add, reorder or categorize assessment columns (quizzes, recitations, major exams, Written Work, Performance Task) by dragging them across the grid.
- **Automatic grade sync.** Scores from digital quizzes and NLP-evaluated essays are filled in the moment a student submits, through the Console's Recorder agent (Feature 5). Physically scanned sheets are added by bulk upload from the OCR pipeline (Feature 4).
- **Real-time computation.** Column totals, term percentages and the running average are recalculated continuously with dynamic weighting, with no custom spreadsheet formulas.
- **College grading scale.** The Overall Grade percentage is converted to the college's own scale (for example 1.00 to 5.00, a 4.0 grade-point scale or letter grades) using bands that the College Admin configures, with passing and failing bands marked.

Workflow

1. Select the class and the term (for example, the Midterm tab) to isolate the view.
2. Configure the structure by dragging column templates into the grid.
3. **Data ingestion.** Cells fill automatically for digital quizzes, or by bulk upload for scanned sheets.
4. **Automated synthesis.** The term percentage, the Overall Grade tab and the analytics are updated instantly.

How it works

- The **State Segregator** filters the full data set to the active tab so only relevant columns are rendered.

- Dropping a column triggers a backend sync that creates a database relationship between that activity and the chosen term.
- The **Analytics Feeder** pushes each student's running average to the retention analytics (Feature 9).
- React (with a drag-and-drop library such as dnd-kit) drives the grid, Python (FastAPI) recalculates class averages on every change, and PostgreSQL stores normalized roster, assessment and E-Classrecord tables so data stays intact even if the browser crashes mid-edit.

Outputs: term-specific ledgers of raw scores and percentages, a combined Overall Grade sheet with its equivalent on the college grading scale, and the percentage values that feed the analytics.

Security and audit. Row-Level Security through FastAPI dependencies checks a teacher's JWT against the classroom owner, so teachers can view and change grades only for their assigned classes. Entries synced by the Recorder are flagged in the database, giving a clear audit trail that separates AI-synced scores from scores the teacher overrode.

Teacher input. For offline or physical assessments, the teacher sets the total items or maximum score when adding a column so percentages are computed correctly.

Feature 8: Class Performance Analytics

Core, statistics-based. This feature saves teachers from tallying and interpreting scores by hand. Once individual results exist, from objective answer-key checking, the essay scorer or the E-Classrecord, they are consolidated into a per-class dashboard. It runs on standard statistics, so its value does not depend on a machine learning model.

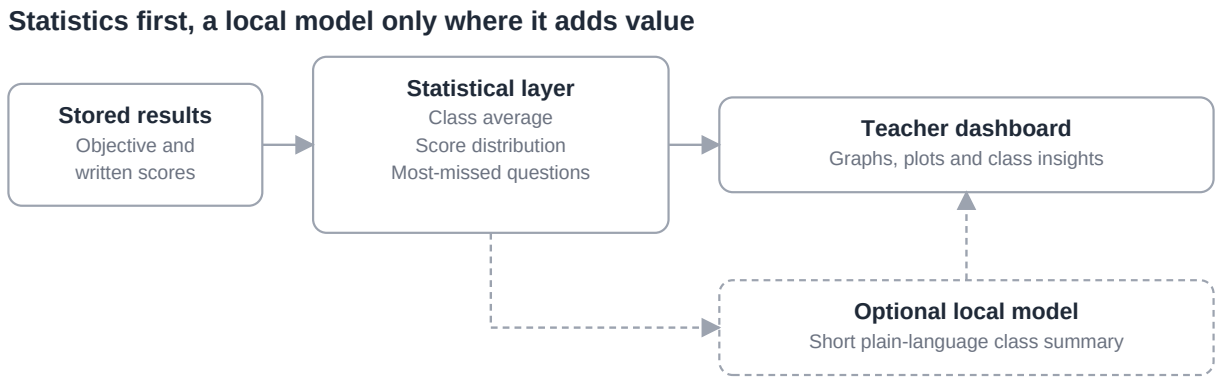


Figure 10. Analytics pipeline: statistics first, optional local model.

- **Per-class dashboard.** Graphs, plots and summary statistics drawn from stored results, covering both objective items and written responses.
- **Baseline metrics.** Class average, score distribution curves, the questions most commonly missed, and how a class compares across exams and activities over time.
- **College-wide view.** Administrators see aggregated performance and system usage across departments, programs and class sections, and can compare the sections of one course taught by different teachers.
- **Optional language-model enhancement.** A local model can add a short plain-language summary of class performance. It follows the free, locally run pattern of the CORNELIA reference application (sentiment and emotion classification, embedding-based topic clustering, keyword extraction), so no paid per-call API is introduced.

Feature 9: Student Retention Analytics

Core, rule-based. Retention Analytics is the early-warning view of AGORA. It turns E-Classrecord scores and attendance logs into a 4-quadrant risk matrix, so teachers and administrators can spot students in danger of failing a course or dropping out well before the term ends.

Two inputs, one 75% rule, four zones

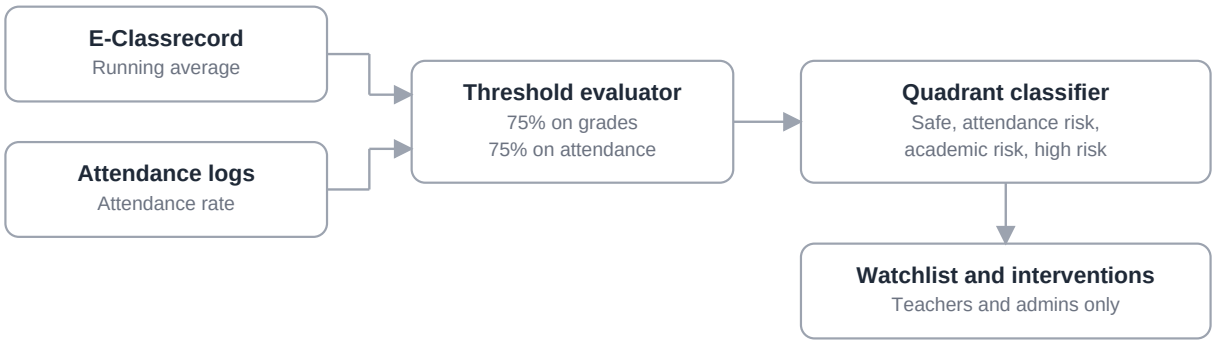


Figure 11. Retention analytics data flow. The 75% baseline is the default and can be set per college.

The engine uses a transparent threshold rule rather than a black-box model, so every classification is easy to justify to students, advisers and counselors. Each student's running average (from the E-Classrecord) and attendance percentage in a class are compared with a baseline for both academics and attendance, 75% by default, and the two results place the student in one of four zones. Because passing marks and attendance rules differ between colleges, each college can set its own baselines.

Grades and attendance against a 75% line give four risk zones

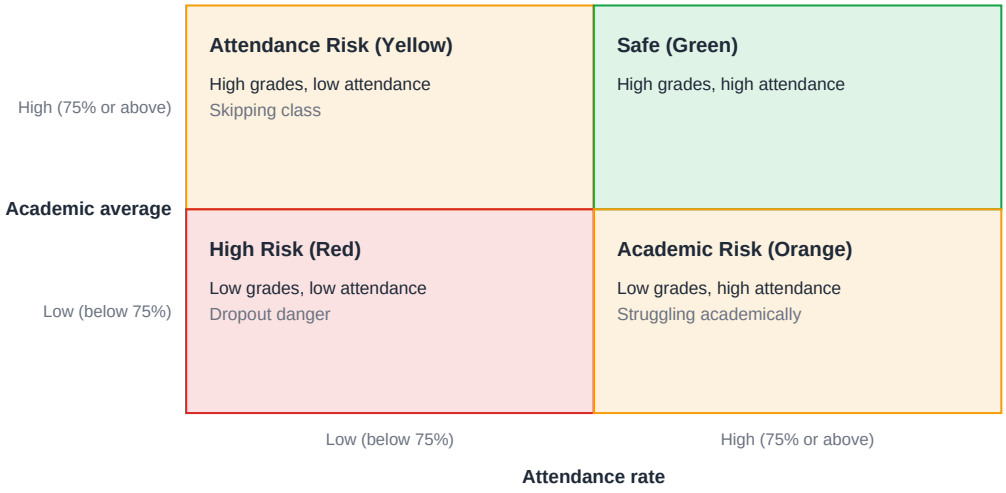


Figure 12. Retention matrix: four zones from a 75% threshold on grades and attendance (the default; configurable per college).

What it does

- **4-quadrant risk matrix.** Plots every student by academic performance against attendance in four color-coded zones, as a scatter plot of the roster.

- **Per-course classification.** A student is classified in each class they are enrolled in, so someone can be safe in one course and at high risk in another.
- **Intervention routing.** Adds quadrant-specific recommendations to student profiles, such as remedial tutoring or a consultation with the course teacher for academic risk, or attendance counseling and a referral to the academic adviser or guidance office for high risk.
- **Real-time updates.** A student's profile shifts as soon as a new assessment score is recorded or an absence is logged.
- **Watchlists.** Filterable lists put high-risk students first for teachers, and College Admins get an aggregated whole-college view, filterable by department and program, to direct resources to struggling courses and programs.

Technologies: Python (FastAPI) calculates percentages and quadrant logic through API endpoints, React with a charting library such as Recharts or Chart.js draws the plots and badges, and PostgreSQL stores the E-Classrecord and attendance data.

Privacy. Risk labels, colors and recommendations are visible only on faculty and admin dashboards, are scoped by role to each teacher's assigned classes, and are hidden from the student portal.

Feature 10: Online Code Compiler / Code Checker

Proposed, prototype built. This feature lets AGORA check coding assessments, which would otherwise fall outside a checker built around text and objective items. Since AGORA targets colleges, including Computer Science and IT programs, it opens the system to a large share of real coursework.

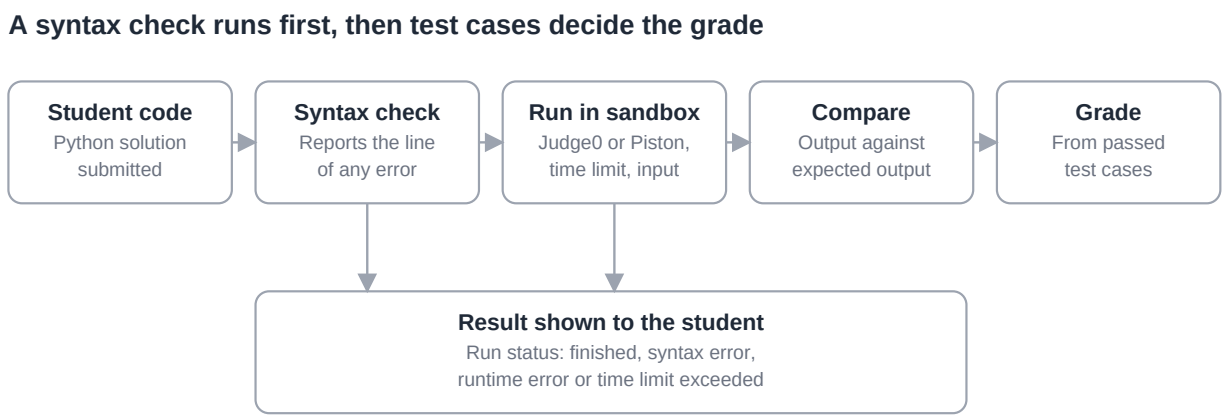


Figure 13. Code checker execution and grading flow.

What it does

- **Code submission.** Students submit code for a problem. Python is the supported language, which keeps the execution setup and grading logic simple.
- **Compile check.** A syntax check runs first and reports the exact line number and message of any syntax error.
- **Execution with input.** The code runs with optional program input (values read by input()), under a time limit, with a cap on returned output.

- **Clear results.** The student sees the output, any errors, the run status (finished, syntax error, runtime error or time limit exceeded) and the run time.
- **Grading.** Submitted code is executed against a defined set of test cases (input and expected-output pairs) attached to each problem, and the output is compared to produce a grade.

Execution engine. Code is run through an established, self-hosted, open-source sandbox engine, Judge0 or Piston. Both are free software with a track record in online-judge platforms and provide resource limits, timeouts and isolation from the host system.

Prototype. A working Flask prototype demonstrates the editor, syntax check, input handling, output and error display, and run timing, with a switchable runner so it can call a self-hosted Piston server.

Feature 11: External API & Data Architecture

Core infrastructure. This feature covers how the checker models are exposed to outside systems and how assessment data is stored internally. Together they determine how a college with its own Virtual Learning Environment (VLE) can adopt AGORA, and how reliably results are retrieved and aggregated for the E-Classrecord and analytics.

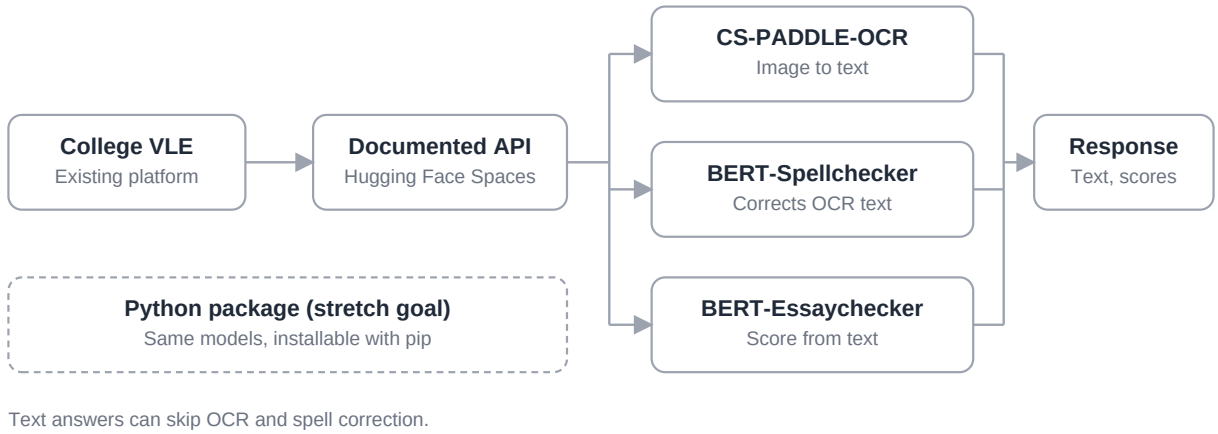


Figure 14. How an external platform uses the hosted models.

Model API

- **Three models, one service.** CS-PADDLE-OCR, BERT-Spellchecker and BERT-Essaychecker are exposed as a separate, callable service that a college's existing platform can integrate with, instead of a full replacement it would need to migrate to.
- **Hosted on Hugging Face Spaces,** a free hosting option for machine learning models.
- **Documented API.** Other platforms send scanned images or text responses and receive extracted text, corrected text and scores back, without adopting the rest of AGORA.
- **Python package (stretch goal).** The models may also be published as a package that can be installed with pip.

Data architecture

- **Relational storage.** Submissions, scores, feedback, override records and statistical summaries are stored across PostgreSQL and MySQL, giving a structured, queryable base for individual results and class-level insight.
- **Core records:** the academic catalog (departments, programs, courses and course outcomes), classes and enrollments (the rosters), assessments (questions, answer keys and signed links), submissions (responses, scores and AI feedback), lesson plans, console prompts and outputs, the E-Classrecord, and retention analytics.
- **Reliability.** Automatic daily backups of the PostgreSQL database and recovery from interruptions without losing stored assessment data.

Application stack. A React web front end, backend services in Python (FastAPI) and Node.js, and local AI inference through Ollama, with all modules reaching the same data layer.

This split keeps each layer, models behind an API, data in a relational database, and the classroom and checking features on top, independently testable and replaceable.

System Architecture

Users reach every module through one React web app and one application server, which calls local AI, a hosted model API, a code engine and two relational databases.

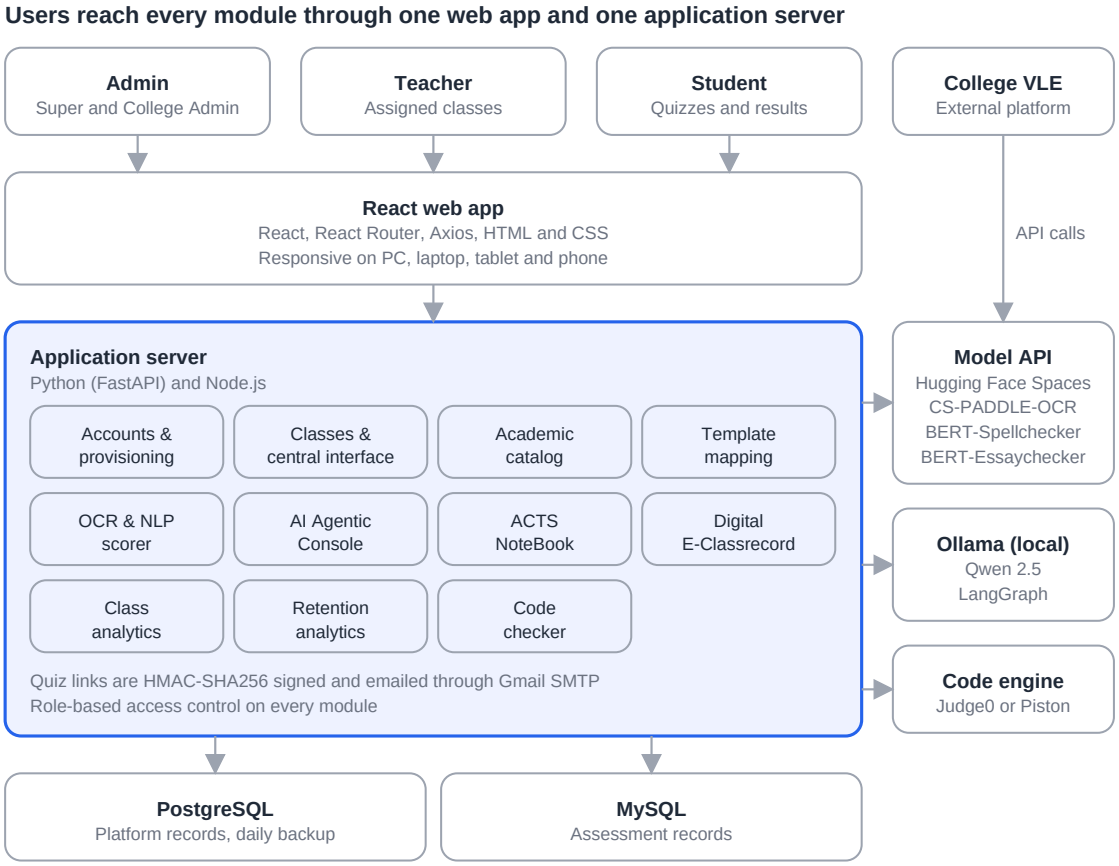


Figure 15. System architecture. External college platforms call the hosted model API directly, without using the rest of AGORA.

Database Design

The merged AGORA database adapts the original school-management schema (11 tables) to colleges. Ten of its tables are carried over (nine of them revised for college use), the `sections` table is removed, and 34 new tables are added: 28 for the AGORA features and 6 for the college structure. That makes 44 tables in total. It targets PostgreSQL and ships as `agora_schema.sql`, which keeps the original tables, revised, and adds the new enums, tables, indexes, triggers and a sample academic year with its semesters and their four grading terms. The migration script and its integrity tests are to be re-run against this college revision.

Main design decisions

- **The classroom is the existing classes table**, now a course offered in one semester by one teacher, with a section code, schedule and room. Templates, assessments, grade components, attendance and analytics all reference it, so access can be scoped to the assigned teacher.
- **Enrollment is per course.** A student is linked to each class they take through enrollments, with a status such as enrolled, dropped, withdrawn or completed. This replaces the fixed section roster, and the sections table is removed.
- **The academic catalog is data.** Departments, programs, courses and course outcomes are tables, so each college keeps its own catalog, and a semester sits between the academic year and the four grading terms.
- **The review trail is kept.** An answer stores the raw OCR text, the corrected text and the teacher-confirmed text, plus the predicted and final score, so teacher overrides are visible and the model output is never lost.
- **Integrity is enforced by the database.** Composite foreign keys make sure an answer's question and submission belong to the same assessment, and a college can have only one current semester.
- **E-Classrecord scores are auditable.** Each score records its source and keeps the original AI score when a teacher overrides it.
- **Weights, scales and thresholds are data, not code.** Grade component weights are stored per class and term (the application checks they add up to 100), and each college stores its own grading scale and retention baselines.
- **Secrets and files stay out of the tables.** Invite and assessment-link tokens are stored as hashes, and lecture files are referenced only, never stored.

The tables are grouped into seven diagrams. Each group's tables are listed below, and the entity relationship diagrams follow, one per page.

Identity and provisioning (carried-over tables, plus account invites and bulk-upload log)

Table	Status	Purpose
colleges	Revised	Participating colleges and universities, added by the Super Admin. Renamed from schools, with college_code and college_type.
users	Revised	Login accounts for all four roles. school_id becomes college_id. The column must_change_password supports the planned forced reset on first login.

Table	Status	Purpose
super_admins, college_admins, teachers, students	Revised	Role profiles keyed by user_id. A composite key guarantees each profile belongs to a user with that role. super_admins is unchanged; school_admins is now college_admins; teachers gain department_id and academic_rank; students gain program_id and year_level.
account_invites	New	Short-lived, single-use invite links, stored as hashes (Feature 1).
teacher_bulk_uploads	New	Log of each College Admin bulk upload: rows created, rows failed and errors (Feature 1).

College setup and classes (carried-over tables, plus the academic structure)

Table	Status	Purpose
academic_years	Revised	Academic years of a college, such as 2026-2027. Renamed from school_years. The current flag moves to semesters.
semesters	New (college)	A term inside an academic year: first semester, second semester or summer, with dates. A college can have only one current semester.
departments	New (college)	Academic departments of a college. Teachers belong to a department and each course is offered by one.
programs	New (college)	Degree programs offered by a department, with degree level and years to complete. Each student belongs to one program.
courses	Revised	The course catalog: code, title, units and type (lecture, laboratory or both). Replaces subjects.
course_outcomes	New (college)	Learning outcomes of a course (for example CLO1) with a target Bloom's level. They can be loaded as the learning goals of the ACTS NoteBook (Feature 6).
classes	Revised	A course offered in one semester and taught by one teacher, with a section code, schedule, room and capacity. This is the classroom of Feature 2, and everything below hangs off it.
enrollments	Revised	A student's enrollment in a class, with its status. This is the class roster, and it replaces the section-based enrollment.

Templates, assessments and questions

Table	Status	Purpose
answer_sheet_templates	New	Reusable sheet layout saved in a classroom (Feature 3). It can be duplicated to the teacher's other classes for the same course.
template_regions	New	One region of the sheet: question type, table or cells layout, and the aligned matrix geometry.
template_answer_keys	New	Answer key per item of a region, for objective checking without a model.
assessments	New	A quiz or exam for a class: delivery (digital or scanned), status, deadline and attempts (Features 4, 5).
questions	New	Items of an assessment with type, prompt, choices, answer key and points.

Table	Status	Purpose
rubric_criteria	New	Teacher-written rubric criteria and point values for essay and short-answer questions (Feature 4).
coding_problems	New	A coding question's statement, language and time limit (Feature 10).
code_test_cases	New	Input and expected-output pairs used to grade code.
assessment_links	New	Per-student HMAC-SHA256 secured link, stored as a token hash with expiry and sent time (Feature 5).

Submissions and scoring

Table	Status	Purpose
submissions	New	One attempt by a student: digital, scanned or typed by the teacher, with status, total score and AI feedback.
answers	New	Per-question answer. Keeps raw OCR text, spell-corrected text, teacher-confirmed final text, mechanics and content sub-scores, predicted and final score, low-confidence flag and reviewer.
rubric_results	New	Per-criterion rubric match: best sentence, similarity, verdict (met, partial, not met) and points awarded.
code_runs	New	Each code run: status, output, run time and passed test cases.

Grades

Table	Status	Purpose
grading_terms	New	Prelim, Midterm, Pre-Final and Final for a semester (Feature 7).
grade_components	New	Weighted components per class and term, such as Written Work and Performance Task.
eclass_columns	New	Assessment columns of the grid, ordered by drag and drop and optionally linked to an assessment.
eclass_scores	New	A student's score in a column, with its source (AI-synced, OCR upload, manual, override) and the original AI score for audit.
grade_scale_bands	New (college)	The college's grading scale: percentage ranges mapped to a grade label and grade point (for example 1.00 to 5.00 or letter grades), with passing bands marked. Used to convert the Overall Grade (Feature 7).

Attendance and analytics

Table	Status	Purpose
attendance_records	New	Daily attendance per class, the second input of retention analytics (Feature 9).
retention_settings	New (college)	Per-college baselines for grades and attendance (75% by default) read by the quadrant classifier (Feature 9).

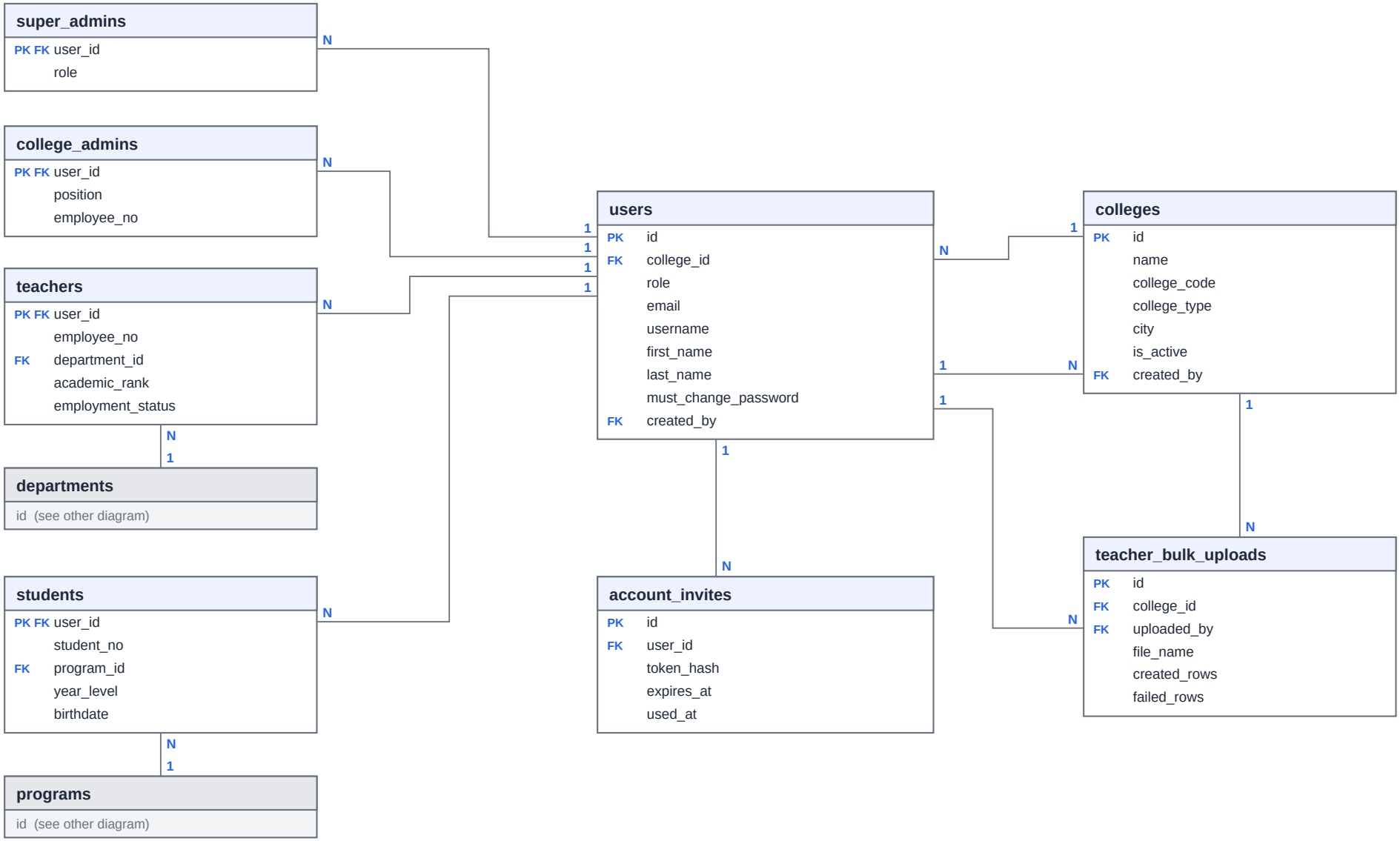
Table	Status	Purpose
retention_analytics	New	Per student and class: running average, attendance rate, quadrant, risk score and intervention.
class_performance_snapshots	New	Stored statistics per class or assessment: mean, median, spread, extremes and per-item miss rates (Feature 8).

AI services and platform

Table	Status	Purpose
lesson_plans	New	ACTS NoteBook output: topics, references to uploaded lecture files (never the files), generated OBE content and export links (Feature 6).
teacher_obe_preferences	New	Saved OBE preferences injected by the Context Assembler node.
agent_prompts	New	Agentic Console runs: the teacher's request, model settings, agent outputs and status (Feature 5).
api_clients	New	Outside platforms allowed to call the hosted model API, with hashed keys (Feature 11).
api_requests	New	Usage log of API calls per client.
notifications	New	Central Interface notifications per user (Feature 2).

Changes to the original schema. The 11 original tables are carried over except `sections`, which is removed. schools becomes colleges, school_admins becomes college_admins, school_years becomes academic_years and subjects becomes courses, and every school_id column becomes college_id. teachers gain department_id and academic_rank. students gain program_id and year_level, and no longer store a guardian. classes now means a course offered in a semester (it gains semester_id, course_id and section_code, and loses section_id). enrollments now link a student to a class instead of to a section. users keeps the must_change_password column added earlier. Status values: Revised means an original table adapted for colleges; New means added for the AGORA features; New (college) means added for the college structure.

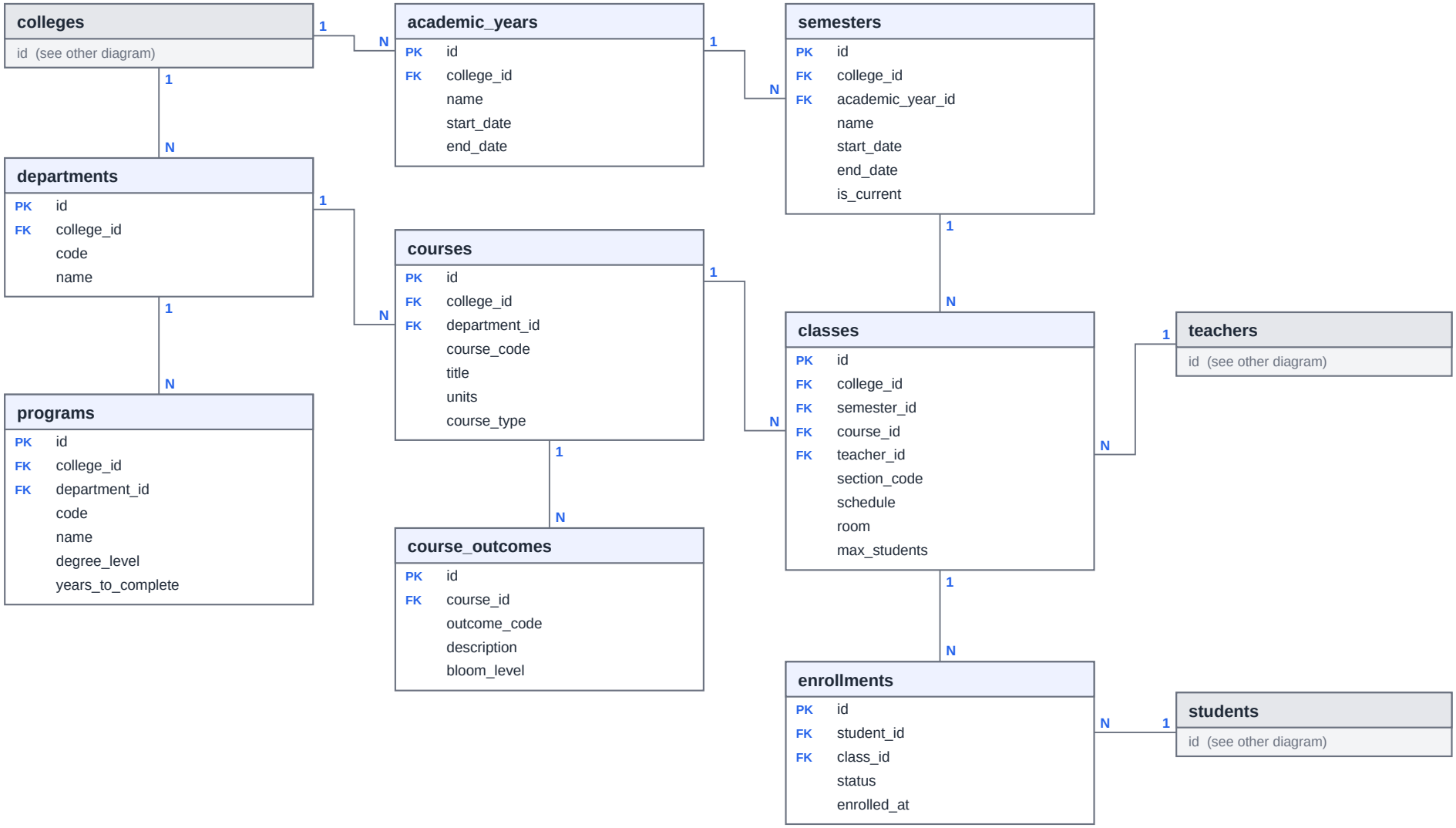
Entity relationship diagram 1: Identity and provisioning



users.created_by also points back to users (self-reference, not drawn). N = many, 1 = one.

Figure 16. Entity relationship diagram: identity and provisioning. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 2: College setup and classes



college_id on semesters, programs, courses and classes also references colleges (not drawn). teachers.department_id and students.program_id are drawn in diagram 1. N = many, 1 = one.

Figure 17. Entity relationship diagram: college setup and classes. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 3: Templates, assessments and questions

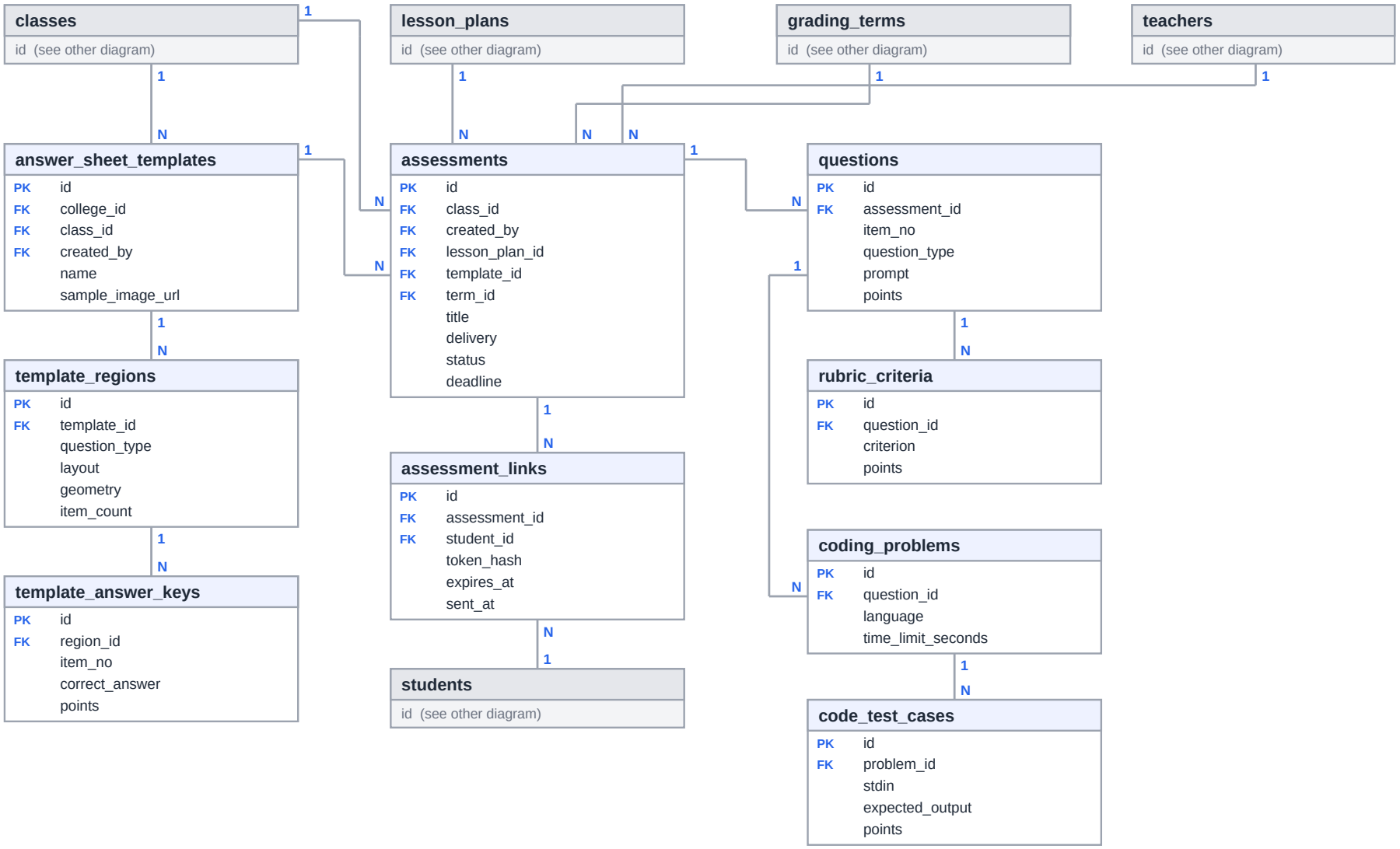


Figure 18. Entity relationship diagram: templates, assessments and questions. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 4: Submissions and scoring

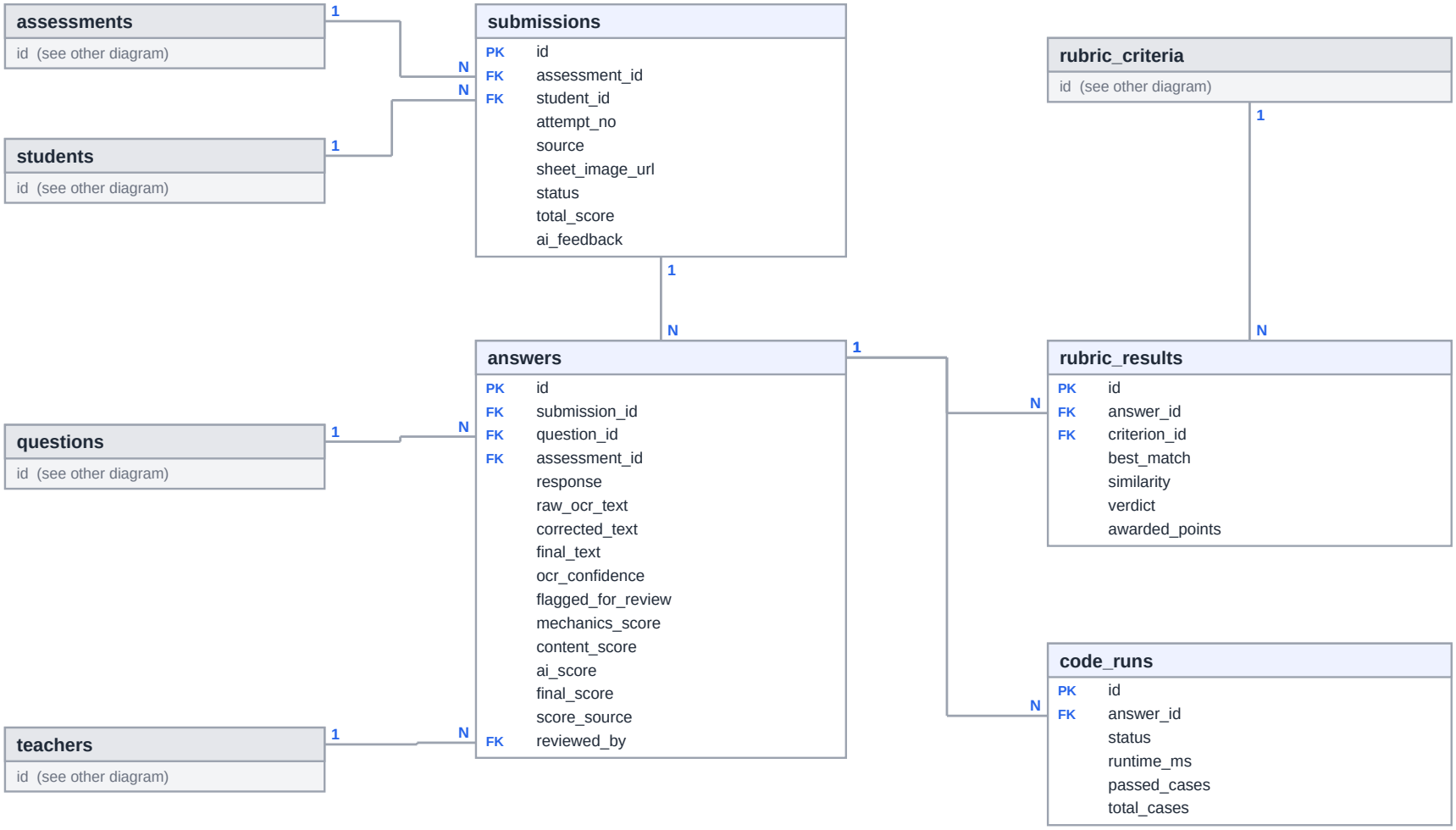


Figure 19. Entity relationship diagram: submissions and scoring. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 5: Grades

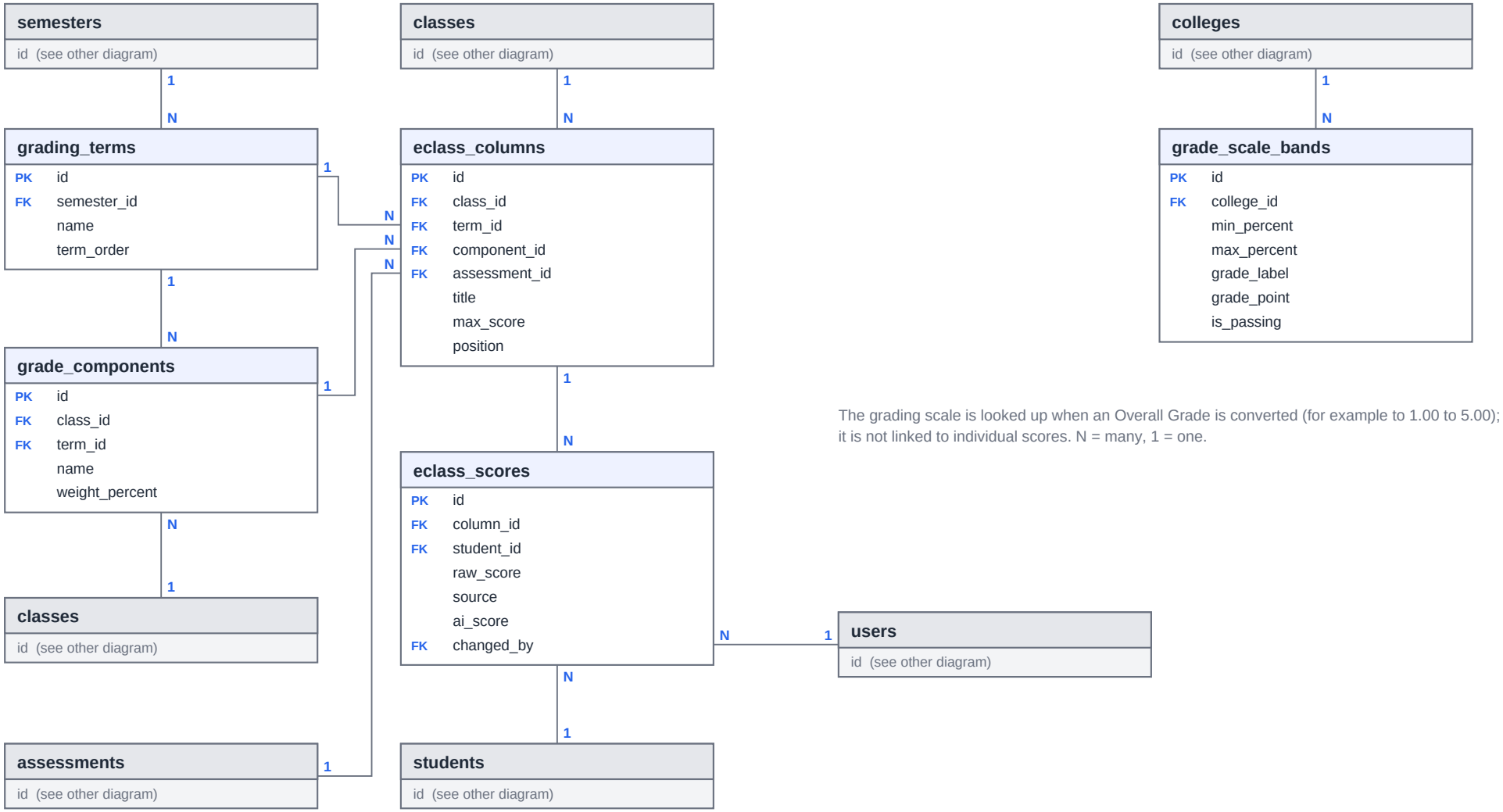
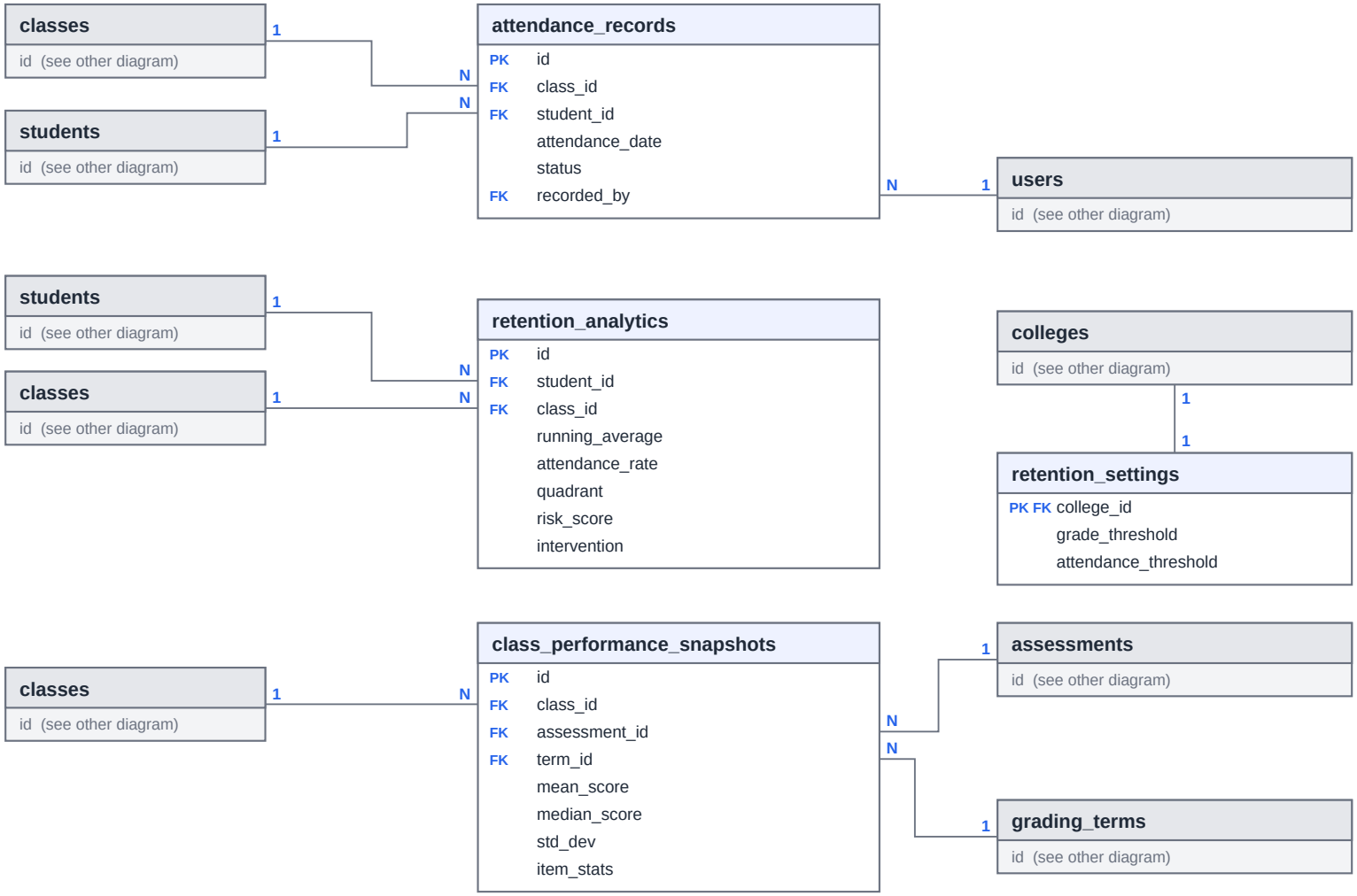


Figure 20. Entity relationship diagram: grades. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 6: Attendance and analytics



retention_settings holds the baselines (75% by default) that the quadrant classifier reads; it is looked up, not linked to each row. N = many, 1 = one.

Figure 21. Entity relationship diagram: attendance and analytics. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 7: AI services and platform

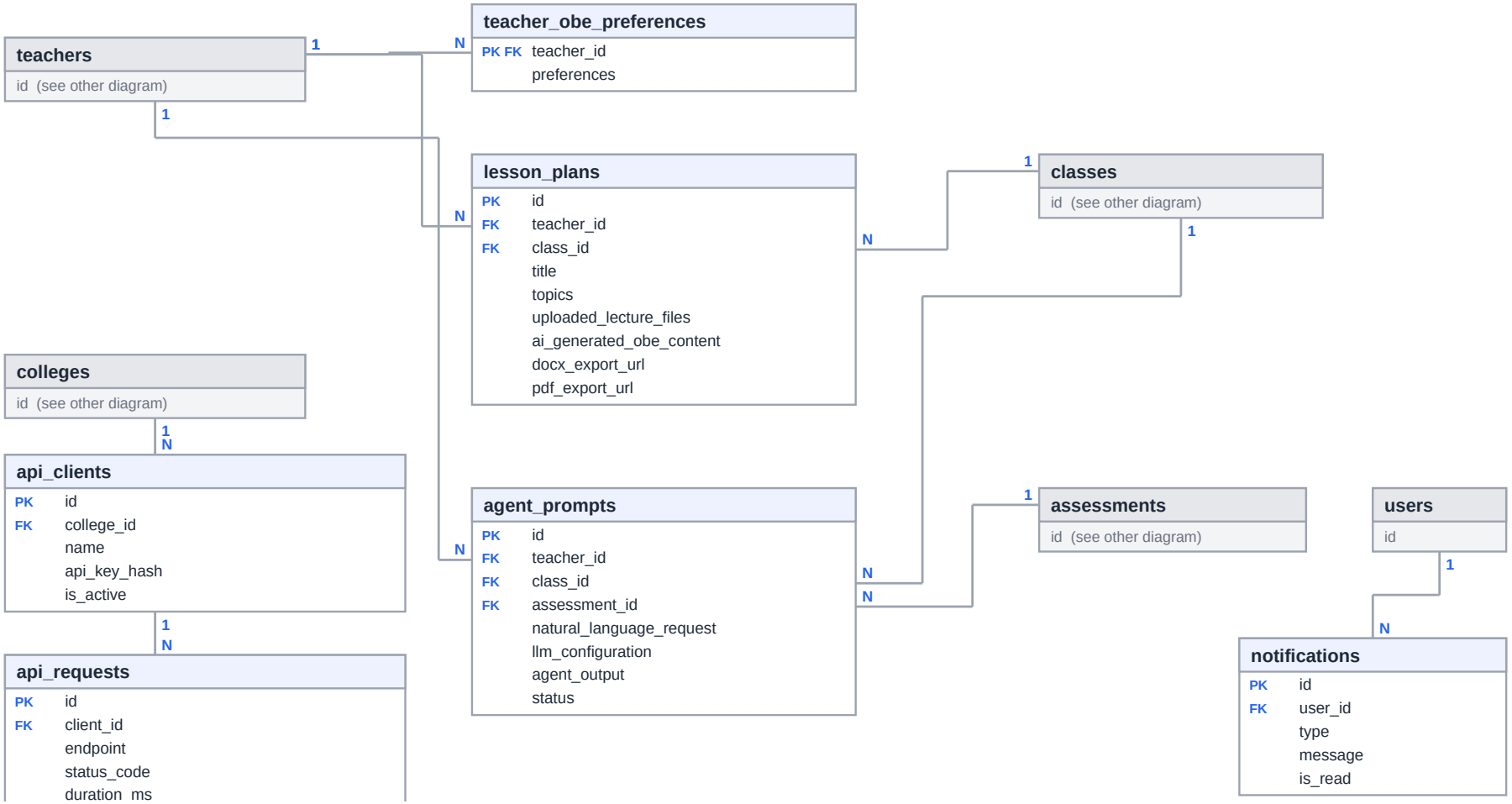


Figure 22. Entity relationship diagram: ai services and platform. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.